

Foundations of Flash attention

[Flash attention]

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1.0 Content Block

In deep learning, the transformer is a family of artificial neural network architectures based on the multi-head attention mechanism, in which text is converted to numerical representations called tokens, and each token is converted into a vector via lookup from a word embedding table. At each layer, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished. Because self-attention alone is permutation-invariant, transformers inject positional information, typically through positional encodings or learned positional embeddings, so token order can affect the output. Transformers have the advantage of having no recurrent units, therefore requiring less training time than earlier recurrent neural architectures (RNNs) such as long short-term memory (LSTM). Later variations have been widely adopted for training large language models (LLMs) on large (language) datasets. Modern transformer designs are commonly grouped into encoder-only, decoder-only, and encoder-decoder variants, depending on whether they are optimized for representation learning, autoregressive generation, or conditional sequence-to-sequence tasks.

2.0 Content Block

A decoder consists of an embedding layer, followed by multiple decoder layers, followed by an un-embedding layer. Each decoder consists of three major components: a causally masked self-attention mechanism, a cross-attention mechanism, and a feed-forward neural network. The decoder functions in a similar fashion to the encoder, but an additional attention mechanism is inserted which instead draws relevant information from the encodings generated by the encoders. This mechanism can also be called the encoder–decoder attention. Like the first encoder, the first decoder takes positional information and embeddings of the output sequence as its input, rather than encodings. The transformer must not use the current or future output to predict an output, so the output sequence must be partially masked to prevent this reverse information flow. This allows for autoregressive text generation. For decoding, all-to-all attention is inappropriate, because a token cannot attend to tokens not yet generated. Thus, the self-attention module in the decoder is causally masked. In contrast, the cross-attention mechanism attends to the output vectors of the encoder, which is computed before the decoder starts decoding. Consequently, there is no need for masking in the cross-attention mechanism. Schematically, we have: $H' = \text{MaskedMultiheadAttention}(H, H, H)$ $\text{DecoderLayer}(H) = \text{FFN}(\text{MultiheadAttention}(H', H^E, H^E))$
$$\begin{aligned} H' &= \text{MaskedMultiheadAttention}(H, H, H) \\ \text{DecoderLayer}(H) &= \text{FFN}(\text{MultiheadAttention}(H', H^E, H^E)) \end{aligned}$$
 where H^E is the matrix with rows being the output vectors from the encoder. The last decoder is followed by a final un-embedding layer to produce the output probabilities over the vocabulary. Then, one of the tokens is sampled according to the probability, and the decoder can be run again to produce the next token, etc., autoregressively generating output text.

3.0 Content Block

$\text{MultiQueryAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(XW_i Q, XW_i K, XW_i V)) W_O$
$$\text{MultiQueryAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(XW_i Q, XW_i K, XW_i V)) W_O$$

$$[n_{\text{heads}}]]\text{left}(\text{Attention})(XW_i^Q, XW^K, XW^V)\text{right})W^O\}$$

4.0 Content Block

KV caching When an autoregressive transformer is used for inference, such as generating text, the query vector is different at each step, but the already-computed key and value vectors are always the same. The KV caching method saves the computed key and value vectors at each attention block, so that they are not recomputed at each new token. PagedAttention applies memory paging to KV caching. If a transformer is used with a baked-in prompt, such as ["You are a customer support agent..."], then the key and value vectors can be computed for the prompt, and saved on disk. The saving in compute is significant when the model is used for many short real-time interactions, such as in online chatbots. In general, when a user uses an autoregressive transformer to generate a continuation to a sequence of tokens, the model would first perform a forward-pass on this sequence, whereby the KV caches over this sequence are computed. This is called prefilling. Hyperscalers serving large Transformer models may use disaggregated inference, wherein prefilling and decoding are performed on separately specialized hardware.

5.0 Content Block

$f(t) = (e^{it/r^k})_{k=0,1,\dots,\frac{d}{2}-1}$ $\{displaystyle f(t)=\left(e^{it/r^k}\right)_{k=0,1,\ldots,\frac{d}{2}-1}$ where $r = N^{2/d}$ $\{displaystyle r=N^{2/d}\}$. The main reason for using this positional encoding function is that using it, shifts are linear transformations: $f(t + \Delta t) = \text{diag}(f(\Delta t))f(t)$ $\{displaystyle f(t+\Delta t)=\mathrm{diag}(f(\Delta t))f(t)\}$ where $\Delta t \in \mathbb{R}$ $\{displaystyle \Delta t\in\mathbb{R}\}$ is the distance one wishes to shift. This allows the transformer to take any encoded position, and find the encoding of the position n -steps-ahead or n -steps-behind, by a matrix multiplication. By taking a linear sum, any convolution can also be implemented as linear transformations: $\sum_j c_j f(t + \Delta t_j) = (\sum_j c_j \text{diag}(f(\Delta t_j)))f(t)$ $\{displaystyle \sum_j c_j f(t+\Delta t_j)=\left(\sum_j c_j\right)\mathrm{diag}(f(\Delta t_j))\text{right}f(t)\}$ for any constants c_j $\{displaystyle c_j\}$. This allows the transformer to take any encoded position and find a linear sum of the encoded locations of its neighbors. This sum of encoded positions, when fed into the attention mechanism, would create attention weights on its neighbors, much like what happens in a convolutional neural network language model. In the author's words, "we hypothesized it would allow the model to easily learn to attend by relative position." In typical implementations, all operations are done over the real numbers, not the complex numbers, but since complex multiplication can be implemented as real 2-by-2 matrix multiplication, this is a mere notational difference.

6.0 Content Block

Alternative attention graphs The standard attention graph is either all-to-all or causal, both of which scales as $O(N^2)$ $\{displaystyle O(N^2)\}$ where N $\{displaystyle N\}$ is the number of tokens in a sequence. Reformer (2020) reduces the computational load from $O(N^2)$ $\{displaystyle O(N^2)\}$ to $O(N \ln N)$ $\{displaystyle O(N\ln N)\}$ by using locality-sensitive hashing and reversible layers. Sparse attention uses attention graphs that grows slower than $O(N^2)$ $\{displaystyle O(N^2)\}$. For example, BigBird (2020) uses random small-world networks which grows as $O(N)$ $\{displaystyle O(N)\}$. Ordinary transformers require a memory size that is quadratic in the size of the context window. Attention-free transformers reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.